Capataz AI — Plan de Desarrollo: Fase Operativa

Documento Técnico para Equipo de Desarrollo

Versión: 1.0
Fecha: Febrero 2026
Autor: Luis Mendez — Terrain Technologies SAPI de CV
Stack principal: Python + FastAPI
Alcance: Capataz AI Operativo (Etapa 1 — sin componentes fintech)

---

1. Visión del Producto

1.1 ¿Qué es Capataz AI Operativo?

Capataz AI Operativo es un asistente de inteligencia artificial diseñado para ayudar a dueños de micro y pequeños negocios a organizar su operación diaria. No es un ERP, no es un CRM, no es un POS. Es un **compañero operativo** que entiende el contexto del negocio y acompaña al dueño en su día a día a través de canales que ya usa (WhatsApp, SMS, app móvil).

1.2 ¿Qué NO es en esta etapa?

- **No es una plataforma fintech.** No hay scoring crediticio, no hay integración con bancos, no hay crédito.
- **No es un sistema de vigilancia.** No monitorea empleados para reportar a terceros.
- **No es un ERP completo.** No maneja contabilidad, facturación ni nómina.
- **No incluye hardware (CapatazX).** No hay wearables, cámaras ni sensores IoT en esta fase.

1.3 Principio rector

> "El pequeño negocio compra resultados, no software."

Todo lo que se construya debe pasar esta prueba: **¿el dueño del negocio siente que esto le ahorra tiempo o le da más ventas HOY?** Si la respuesta es no, no se construye en esta etapa.

1.4 Estrategia de etapas

Etapa	Nombre	Objetivo	Dependencia
1	**Capataz Operativo**	Demostrar valor diario, generar adopción	Ninguna
2	Capataz Data/Analytics	Insights y métricas a partir del uso acumulado	Etapa 1 validada
3	Capataz Fintech	Scoring crediticio y acceso a crédito	Etapa 2 con datos suficientes

Este documento cubre exclusivamente la Etapa 1.

---

2. Usuarios y Segmentos

2.1 Perfil del usuario principal

Atributo	Descripción
Quién	Dueño/a o gerente de micro o pequeño negocio
Giro	Restaurantes, comercios minoristas, talleres, negocios de servicios
Ubicación inicial	Atlixco y zona metropolitana de Puebla
Edad típica	28–55 años
Nivel digital	Medio-bajo. Usa WhatsApp y redes sociales, pero no herramientas empresariales
Dispositivo	Smartphone Android (mayoritariamente), algunos iOS
Dolor principal	Desorden operativo: tareas olvidadas, falta de seguimiento, estrés diario
Motivación	Que el negocio funcione mejor sin invertir horas en aprender software
Resistencia	Cualquier cosa que le quite tiempo, se sienta complicada o lo haga sentir vigilado

2.2 Perfil del usuario secundario

Atributo	Descripción
Quién	Empleado o colaborador operativo
Interacción	Recibe tareas, reporta avances, consulta instrucciones
Nivel digital	Bajo. Interactúa solo si es por WhatsApp o algo muy simple
Motivación	Saber qué le toca hacer y que le reconozcan cuando lo hace bien
Resistencia	Sentirse vigilado o controlado

2.3 Roles del sistema

Rol	Permisos
`owner`	CRUD completo de negocio, tareas, empleados, reportes
`manager`	Crear/editar tareas, ver reportes del equipo, gestionar empleados
`employee`	Ver sus tareas asignadas, reportar avance, consultar instrucciones

---

3. Arquitectura de Alto Nivel

3.1 Diagrama conceptual

```
┌──────────────────────────────────────────────────────────────────────┐
│                              CLIENTES                                 │
│                                                                       │
│   ┌──────────┐    ┌──────────────┐    ┌──────────────────┐          │
│   │ WhatsApp │    │  App Móvil   │    │  Web Dashboard   │          │
│   │  (Meta   │    │  (Flutter)   │    │  (Jinja2 +       │          │
│   │Cloud API)│    │              │    │  HTMX)           │          │
│   └──────────┘    └──────────────┘    └──────────────────┘          │
│        │                │                     │                      │
└────────┼────────────────┼─────────────────────┼──────────────────────┘
         │                │                     │
         ▼                ▼                     ▼

┌──────────────────────────────────────────────────────────────────────┐
│                     API GATEWAY / LOAD BALANCER                       │
│                                (nginx )                               │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              ▼

┌──────────────────────────────────────────────────────────────────────┐
│                      FASTAPI APPLICATION                              │
│                                                                       │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐       │
│  │   Auth &     │  │   Business   │  │    Tasks &           │       │
│  │   Users      │  │   Management │  │    Operations        │       │
│  │   Module     │  │   Module     │  │    Module            │       │
│  └──────────────┘  └──────────────┘  └──────────────────────┘       │
│                                                                       │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐       │
│  │ Messaging    │  │  AI Engine   │  │   Notifications      │       │
│  │ & Channels   │  │  (LLM +      │  │   & Reminders        │       │
│  │ Module       │  │  Rules)      │  │   Module             │       │
│  └──────────────┘  └──────────────┘  └──────────────────────┘       │
│                                                                       │
│  ┌──────────────┐  ┌──────────────┐                                  │
│  │  Reports     │  │  Onboarding  │                                  │
│  │  Module      │  │  Module      │                                  │
│  └──────────────┘  └──────────────┘                                  │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              │
                   ┌──────────┼──────────┐
                   ▼          ▼          ▼

   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
   │PostgreSQL│  │  Redis   │  │  S3 /    │  │  vLLM    │
   │  (Main   │  │ (Cache,  │  │  Minio   │  │  Server  │
   │   DB)    │  │  Queue,  │  │ (Files)  │  │ (Qwen3/  │
   │          │  │  Sessions│  │          │  │ Llama on │
   │          │  │          │  │          │  │   GPU)   │
   └──────────┘  └──────────┘  └──────────┘  └──────────┘
```

3.2 Stack tecnológico detallado

Capa	Tecnología	Justificación
Backend API	Python 3.12 + FastAPI	Async nativo, tipado fuerte con Pydantic, docs automáticos (OpenAPI), ecosistema de ML/AI
ORM	SQLAlchemy 2.0 + Alembic	Migraciones versionadas, soporte async, maduro
Base de datos	PostgreSQL 16	JSONB para datos flexibles, full-text search, extensiones (pg_trgm, uuid-ossp)
Cache / Broker	Redis 7	Sesiones, cache de queries, cola de tareas (con Celery o arq)
Cola de tareas	Celery + Redis (o arq)	Envío de notificaciones, procesamiento async de reportes, jobs de IA
Almacenamiento	MinIO (self-hosted) o almacenamiento en VPS	Imágenes, archivos adjuntos, exports de reportes
Mensajería WhatsApp	Meta Cloud API (WhatsApp Business Platform)	Canal principal de interacción
AI / LLM	Groq Cloud pagado (temporal) → vLLM server local con GPUs propias del proyecto (destino)	Groq pagado (sin límites de contexto). Meta: inferencia local al 100% para Sprint 3. GPU 0: modelo de texto 16B+. GPU 1: DeepSeek-OCR para visión. API compatible OpenAI en todos los casos
Autenticación	JWT (access + refresh tokens)	Stateless, compatible con múltiples clientes
Web Dashboard	FastAPI + Jinja2 + Vanilla JS + HTMX + Chart.js	Dashboard propio server-side rendered, sin frameworks JS pesados. Charts con Chart.js, interactividad con HTMX y vanilla JS
App Móvil	Flutter	Canal secundario después de WhatsApp, misma codebase para Android e iOS

Capa	Tecnología	Justificación
Infraestructura	2x HP Z6 G4 (inferencia local) + VPS cloud (app)	Server 1: texto (2x PRO 5000). Server 2: OCR (1x PRO 5000 + slot libre). VPS para FastAPI + PostgreSQL. Deployments con systemd + scripts
CI/CD	GitHub Actions	Tests, lint, deploy automático
Monitoreo	Sentry (errores) + métricas custom en dashboard propio	Observabilidad integrada al dashboard de Capataz, sin Grafana
Testing	pytest + httpx (async)	Tests unitarios e integración

3.3 Estructura del proyecto FastAPI

```
capataz-api/
├── alembic/                    # Migraciones de DB
│   └── versions/
├── app/
│   ├── __init__.py
│   ├── main.py                 # Aplicación FastAPI, startup/shutdown
│   ├── config.py               # Settings con pydantic-settings
│   ├── dependencies.py         # Dependencias inyectables (db, auth, etc.)
│   │
│   ├── auth/                   # Módulo de autenticación
│   │   ├── router.py
│   │   ├── service.py
│   │   ├── schemas.py
│   │   └── utils.py
│   │
│   ├── users/                  # Módulo de usuarios
│   │   ├── router.py
│   │   ├── service.py
│   │   ├── schemas.py
│   │   └── models.py
│   │
│   ├── businesses/             # Módulo de negocios
│   │   ├── router.py
│   │   ├── service.py
│   │   ├── schemas.py
│   │   └── models.py
│   │
│   ├── tasks/                  # Módulo de tareas y operaciones
│   │   ├── router.py
│   │   ├── service.py
│   │   ├── schemas.py
│   │   └── models.py
│   │
│   ├── employees/              # Módulo de empleados/colaboradores
│   │   ├── router.py
│   │   ├── service.py
│   │   ├── schemas.py
│   │   └── models.py
│   │
│   ├── notifications/          # Módulo de notificaciones y recordatorios
│   │   ├── router.py
│   │   ├── service.py
│   │   ├── schemas.py
│   │   └── models.py
│   │
│   ├── messaging/              # Módulo de canales (WhatsApp, SMS)
│   │   ├── router.py           # Webhooks de WhatsApp
│   │   ├── service.py
│   │   ├── whatsapp_client.py
│   │   └── schemas.py
```

```
│   │
│   ├── ai/                     # Módulo de IA / Asistente
│   │   ├── engine.py           # Orquestador de conversación
│   │   ├── prompts.py          # Templates de prompts
│   │   ├── intents.py          # Clasificación de intenciones
│   │   └── context.py          # Manejo de contexto conversacional
│   │
│   ├── reports/                # Módulo de reportes
│   │   ├── router.py
│   │   ├── service.py
│   │   └── schemas.py
│   │
│   ├── onboarding/             # Módulo de onboarding
│   │   ├── router.py
│   │   ├── service.py
│   │   └── flows.py
│   │
│   ├── dashboard/              # Dashboard web (server-side rendered)
│   │   ├── router.py           # Rutas que renderizan HTML
│   │   ├── views.py            # Lógica de vistas
│   │   └── context.py          # Helpers para construir datos de las vistas
│   │
│   ├── db/
│   │   ├── session.py          # Engine y SessionLocal async
│   │   └── base.py             # Base declarativa
│   │
│   └── common/
│       ├── exceptions.py       # Excepciones custom
│       ├── pagination.py       # Paginación estándar
│       └── responses.py        # Schemas de respuesta estándar
│
├── templates/                  # Jinja2 templates (HTML)
│   ├── base.html               # Layout principal
│   ├── components/             # Componentes reutilizables
│   │   ├── sidebar.html
│   │   ├── header.html
│   │   ├── task_card.html
│   │   ├── chart_widget.html
│   │   └── employee_row.html
│   ├── dashboard/
│   │   ├── home.html           # Vista principal del dueño
│   │   ├── tasks.html          # Vista de tareas
│   │   ├── employees.html      # Vista de equipo
│   │   ├── checklists.html     # Vista de checklists
│   │   ├── reports.html        # Vista de reportes
│   │   └── settings.html       # Configuración del negocio
│   └── auth/
│       ├── login.html
│       └── verify.html
│
├── static/                     # Archivos estáticos
```

```
│   ├── css/
│   │   ├── main.css            # Estilos propios (CSS vanilla o con variables)
│   │   └── components.css      # Estilos de componentes
│   ├── js/
│   │   ├── app.js              # JS principal (vanilla)
│   │   ├── charts.js           # Lógica de gráficas (Chart.js)
│   │   ├── htmx.min.js         # HTMX para interactividad sin SPA
│   │   └── utils.js            # Utilidades
│   └── img/
│       └── logo.svg
│
├── tests/
│   ├── conftest.py
│   ├── test_auth/
│   ├── test_tasks/
│   ├── test_messaging/
│   └── test_ai/
│
├── deploy/                     # Scripts de deployment
│   ├── setup.sh                # Setup del server
│   ├── deploy.sh               # Deploy de la app
│   └── vllm.service            # Systemd unit para vLLM
├── pyproject.toml
├── alembic.ini
└── README.md
```

---

4. Modelo de Datos

4.1 Entidades principales

`users`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
phone	VARCHAR(20) UNIQUE	Número de teléfono (login principal)
email	VARCHAR(255) NULL	Email opcional
name	VARCHAR(150)	Nombre completo
password_hash	VARCHAR(255) NULL	Hash bcrypt (null si solo usa WhatsApp)
role	ENUM('owner','manager','employee')	Rol global
is_active	BOOLEAN	Estado activo
preferred_channel	ENUM('whatsapp','sms','app','web')	Canal preferido de comunicación
timezone	VARCHAR(50)	Zona horaria del usuario
language	VARCHAR(5) DEFAULT 'es'	Idioma
created_at	TIMESTAMPTZ	Fecha de creación
updated_at	TIMESTAMPTZ	Última actualización

`businesses`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
owner_id	UUID (FK → users)	Dueño del negocio
name	VARCHAR(200)	Nombre del negocio
business_type	VARCHAR(100)	Giro (restaurante, comercio, taller, etc.)
address	TEXT NULL	Dirección
city	VARCHAR(100)	Ciudad
state	VARCHAR(100)	Estado
phone	VARCHAR(20) NULL	Teléfono del negocio
employee_count	INTEGER	Número aproximado de empleados
operating_hours	JSONB	Horarios de operación
onboarding_completed	BOOLEAN DEFAULT FALSE	Si completó el onboarding
settings	JSONB	Configuraciones del negocio
created_at	TIMESTAMPTZ	Fecha de creación

`business_members`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
business_id	UUID (FK → businesses)	Negocio
user_id	UUID (FK → users)	Usuario
role	ENUM('owner','manager','employee')	Rol dentro del negocio
position	VARCHAR(100) NULL	Puesto (ej: "cocinero", "cajera")
is_active	BOOLEAN	Si sigue activo en el negocio
joined_at	TIMESTAMPTZ	Fecha en que se unió

`tasks`

Campo	Tipo	Descripci
id	UUID (PK)	Identificac único
business_id	UUID (FK → businesses)	Negocio a que pertenece
created_by	UUID (FK → users)	Quien cre la tarea
assigned_to	UUID (FK → users) NULL	A quién se asigna
title	VARCHAR(300)	Título de l tarea
description	TEXT NULL	Descripció detallada
status	ENUM('pending','in_progress','completed','cancelled','overdue')	Estado
priority	ENUM('low','medium','high','urgent')	Prioridad
category	VARCHAR(100) NULL	Categoría (limpieza, inventario compras, etc.)
due_date	TIMESTAMPTZ NULL	Fecha lím
completed_at	TIMESTAMPTZ NULL	Cuándo s completó
recurrence	JSONB NULL	Regla de recurrenci (diario, semanal, etc.)
source	ENUM('manual','ai_suggested','template','recurring')	Origen de tarea

Campo	Tipo	Descripci
metadata	JSONB	Datos adicionale flexibles
created_at	TIMESTAMPTZ	Fecha de creación
updated_at	TIMESTAMPTZ	Última actualizac

`task_comments`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
task_id	UUID (FK → tasks)	Tarea relacionada
user_id	UUID (FK → users)	Quien comentó
content	TEXT	Contenido del comentario
attachments	JSONB NULL	URLs de archivos adjuntos
created_at	TIMESTAMPTZ	Fecha de creación

`checklists`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
business_id	UUID (FK → businesses)	Negocio
name	VARCHAR(200)	Nombre del checklist
description	TEXT NULL	Descripción
category	VARCHAR(100)	Categoría (apertura, cierre, limpieza, inventario)
recurrence	JSONB NULL	Cuándo se activa
is_template	BOOLEAN DEFAULT FALSE	Si es plantilla reutilizable
items	JSONB	Array de items con {text, order, required}
created_at	TIMESTAMPTZ	Fecha de creación

`checklist_completions`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
checklist_id	UUID (FK → checklists)	Checklist completado
completed_by	UUID (FK → users)	Quién lo completó
completion_data	JSONB	Items marcados, notas, fotos
completed_at	TIMESTAMPTZ	Cuándo se completó
completion_time_seconds	INTEGER	Tiempo que tardó en completarlo

`reminders`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
business_id	UUID (FK → businesses)	Negocio
user_id	UUID (FK → users)	A quién va dirigido
message	TEXT	Mensaje del recordatorio
channel	ENUM('whatsapp','sms','push','in_app')	Canal de envío
scheduled_at	TIMESTAMPTZ	Cuándo se envía
recurrence	JSONB NULL	Regla de recurrencia
status	ENUM('scheduled','sent','failed','cancelled')	Estado
sent_at	TIMESTAMPTZ NULL	Cuándo se envió realmente
source	ENUM('manual','ai_suggested','system')	Origen
created_at	TIMESTAMPTZ	Fecha de creación

`conversations`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
business_id	UUID (FK → businesses)	Negocio
user_id	UUID (FK → users)	Usuario que conversa
channel	ENUM('whatsapp','sms','app','web')	Canal
status	ENUM('active','idle','closed')	Estado
context	JSONB	Contexto acumulado de la conversación
last_message_at	TIMESTAMPTZ	Último mensaje
created_at	TIMESTAMPTZ	Inicio de la conversación

`messages`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
conversation_id	UUID (FK → conversations)	Conversación
sender	ENUM('user','assistant','system')	Quién envió
content	TEXT	Contenido del mensaje
message_type	ENUM('text','image','audio','template','interactive')	Tipo
intent	VARCHAR(100) NULL	Intención detectada por IA
metadata	JSONB	Datos adicionales (WhatsApp message_id, etc.)
created_at	TIMESTAMPTZ	Fecha de envío

`daily_summaries`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
business_id	UUID (FK → businesses)	Negocio
date	DATE	Fecha del resumen
tasks_created	INTEGER	Tareas creadas ese día
tasks_completed	INTEGER	Tareas completadas
tasks_overdue	INTEGER	Tareas vencidas
checklists_completed	INTEGER	Checklists completados
active_employees	INTEGER	Empleados activos ese día
ai_interactions	INTEGER	Interacciones con el asistente
highlights	JSONB	Puntos destacados generados por IA
generated_at	TIMESTAMPTZ	Cuándo se generó

`activity_log`

Campo	Tipo	Descripción
id	UUID (PK)	Identificador único
business_id	UUID (FK → businesses)	Negocio
user_id	UUID (FK → users) NULL	Usuario que realizó la acción
action	VARCHAR(100)	Tipo de acción (task.created, checklist.completed, etc.)
entity_type	VARCHAR(50)	Tipo de entidad afectada
entity_id	UUID	ID de la entidad afectada
details	JSONB	Detalles de la acción
created_at	TIMESTAMPTZ	Cuándo ocurrió

4.2 Índices recomendados

```
-- Búsquedas frecuentes
CREATE INDEX idx_tasks_business_status ON tasks(business_id, status);
CREATE INDEX idx_tasks_assigned_status ON tasks(assigned_to, status);
CREATE INDEX idx_tasks_due_date ON tasks(due_date) WHERE status NOT IN
('completed', 'cancelled');
CREATE INDEX idx_reminders_scheduled ON reminders(scheduled_at) WHERE status =
'scheduled';
CREATE INDEX idx_messages_conversation ON messages(conversation_id, created_at
DESC);
CREATE INDEX idx_activity_business_date ON activity_log(business_id, created_at
DESC);
CREATE INDEX idx_checklist_completions_date ON
checklist_completions(checklist_id, completed_at DESC);
CREATE INDEX idx_daily_summaries_lookup ON daily_summaries(business_id, date
DESC);
```

---

5. Módulos y Funcionalidades Detalladas

5.1 Módulo de Autenticación y Usuarios (`auth/`, `users/`)

Objetivo: Registro e inicio de sesión simples, priorizando el teléfono como identidad principal.

> **Decisión sobre Isora (auth existente de Terrain):** Después de revisar el código fuente de Isora, **NO se reutiliza como base**. Isora está construido sobre Flask (sync, WSGI, Flask-Login, Flask-WTF) y Capataz es FastAPI (async, ASGI, JWT). Portarlo sería una reescritura completa, no un refactor.
>
> **Lo que sí se porta como concepto (no como código):**
>
> - Fernet encryption para datos PII del usuario (reimplementar en ~20 líneas)
> - Patrón de LoginAttempt + lockout (5 intentos → 15 min bloqueo)
> - Estructura del AuditLog (user_id, action, resource_type, resource_id, ip, metadata, status)
> - @audit decorator adaptado como FastAPI dependency o middleware
>
> **Lo que Capataz NO necesita de Isora:**
>
> - 2FA/TOTP con authenticator — Capataz usa OTP por WhatsApp/SMS
> - Flask-Login / server sessions — Capataz usa JWT para API + session cookie para dashboard
> - Flask-WTF / CSRF forms — Capataz usa HTMX, no forms tradicionales
> - Flask-Mail — Capataz notifica por WhatsApp, no por email
> - Flask-Talisman — FastAPI tiene su propio middleware para security headers

Funcionalidades:

- Registro con número de teléfono + código OTP vía WhatsApp/SMS
- Login con OTP (sin contraseña para el flujo principal)
- Login alternativo con email + contraseña (para dashboard web)
- JWT con access token (15 min) + refresh token (30 días)
- Gestión de perfil básico
- Recuperación de acceso por OTP

Endpoints principales:

Método	Ruta	Descripción
POST	`/auth/request-otp`	Solicitar código OTP
POST	`/auth/verify-otp`	Verificar OTP y obtener tokens
POST	`/auth/refresh`	Renovar access token
POST	`/auth/login`	Login email + contraseña (web)
GET	`/users/me`	Perfil del usuario autenticado
PATCH	`/users/me`	Actualizar perfil

5.2 Módulo de Negocios (`businesses/`)

Objetivo: Gestionar el negocio del usuario, configurarlo y vincular empleados.

Funcionalidades:

- Crear negocio (uno por dueño en MVP, multi-negocio después)
- Configurar giro, horarios, preferencias
- Invitar empleados por número de teléfono
- Gestionar miembros del equipo (roles, activar/desactivar)
- Configuraciones del negocio (categorías de tareas, horarios, etc.)

Endpoints principales:

Método	Ruta	Descripción
POST	`/businesses`	Crear negocio
GET	`/businesses/{id}`	Obtener detalle del negocio
PATCH	`/businesses/{id}`	Actualizar información
POST	`/businesses/{id}/members`	Invitar miembro
GET	`/businesses/{id}/members`	Listar miembros
PATCH	`/businesses/{id}/members/{member_id}`	Actualizar rol/estado
DELETE	`/businesses/{id}/members/{member_id}`	Remover miembro

5.3 Módulo de Tareas y Operaciones (`tasks/`)

Objetivo: El corazón del producto. Crear, asignar, dar seguimiento y completar tareas operativas.

Funcionalidades:

- CRUD de tareas con asignación, prioridad, fecha límite y categoría
- Cambio de estado con timestamps (pending → in_progress → completed)
- Tareas recurrentes (diarias, semanales, mensuales) con generación automática
- Tareas sugeridas por IA basadas en el giro del negocio
- Comentarios y adjuntos en tareas
- Filtros por estado, asignado, categoría, fecha
- Detección automática de tareas vencidas (job en background)

Endpoints principales:

Método	Ruta	Descripción
POST	`/businesses/{id}/tasks`	Crear tarea
GET	`/businesses/{id}/tasks`	Listar tareas (con filtros)
GET	`/businesses/{id}/tasks/{task_id}`	Detalle de tarea
PATCH	`/businesses/{id}/tasks/{task_id}`	Actualizar tarea
POST	`/businesses/{id}/tasks/{task_id}/complete`	Marcar como completada
POST	`/businesses/{id}/tasks/{task_id}/comments`	Agregar comentario
GET	`/businesses/{id}/tasks/summary`	Resumen de tareas (hoy, pendientes, vencidas)
POST	`/businesses/{id}/tasks/generate-suggestions`	Sugerencias de IA

5.4 Módulo de Checklists (`checklists/`)

Objetivo: Rutinas operativas estandarizadas (apertura, cierre, limpieza, inventario rápido).

Funcionalidades:

- Crear checklists con items ordenados
- Plantillas predefinidas por giro (restaurante, tienda, taller)
- Completar checklist item por item con notas opcionales y fotos

- Checklists recurrentes que se activan automáticamente
- Historial de completaciones

Endpoints principales:

Método	Ruta	Descripción
POST	`/businesses/{id}/checklists`	Crear checklist
GET	`/businesses/{id}/checklists`	Listar checklists
GET	`/businesses/{id}/checklists/templates`	Plantillas disponibles por giro
POST	`/businesses/{id}/checklists/{cl_id}/complete`	Registrar completación
GET	`/businesses/{id}/checklists/{cl_id}/history`	Historial de completaciones

5.5 Módulo de Recordatorios y Notificaciones (`notifications/`, `reminders/`)

Objetivo: Que nada se olvide. Recordatorios proactivos que llegan al canal correcto en el momento correcto.

Funcionalidades:

- Recordatorios manuales (el dueño programa uno)
- Recordatorios automáticos vinculados a tareas (antes de vencer, al vencer)
- Recordatorios del sistema (resumen matutino, cierre del día)
- Envío multi-canal (WhatsApp prioritario, push como fallback)
- Recordatorios sugeridos por IA basados en patrones

Lógica de recordatorios automáticos:

Trigger	Cuándo	A quién
Tarea por vencer	2 horas antes del due_date	assigned_to
Tarea vencida	30 min después del due_date	assigned_to + owner
Resumen matutino	Al inicio del horario operativo	owner
Cierre del día	30 min antes de cerrar	owner
Checklist pendiente	Según recurrencia configurada	asignado o owner
Inactividad	Si no hay interacción en 48 horas	owner

Endpoints principales:

Método	Ruta	Descripción
POST	`/businesses/{id}/reminders`	Crear recordatorio manual
GET	`/businesses/{id}/reminders`	Listar recordatorios
DELETE	`/businesses/{id}/reminders/{rem_id}`	Cancelar recordatorio
GET	`/users/me/notifications`	Notificaciones del usuario
PATCH	`/users/me/notifications/{notif_id}/read`	Marcar como leída

5.6 Módulo de Mensajería y Canales (`messaging/`)

Objetivo: Integración con WhatsApp como canal principal de interacción con el asistente.

Funcionalidades:

- Webhook de recepción de mensajes de WhatsApp (Meta Cloud API)
- Envío de mensajes de texto, templates, botones interactivos y listas
- Procesamiento de mensajes de voz (speech-to-text)
- Procesamiento de imágenes enviadas (ej: foto de inventario)
- Gestión de sesiones conversacionales
- Fallback a SMS si WhatsApp no está disponible

Flujo de un mensaje entrante:

```
WhatsApp → Webhook POST /messaging/whatsapp/webhook
  → Validar firma de Meta
  → Identificar usuario por teléfono
  → Cargar/crear conversación
  → Enviar contenido al AI Engine
  → AI Engine procesa intención + contexto
  → Generar respuesta
  → Ejecutar acción si aplica (crear tarea, completar checklist, etc.)
  → Enviar respuesta por WhatsApp
  → Guardar mensaje en DB
```

Endpoints principales:

Método	Ruta	Descripción
GET	`/messaging/whatsapp/webhook`	Verificación del webhook de Meta
POST	`/messaging/whatsapp/webhook`	Recepción de mensajes
POST	`/messaging/send`	Envío programático de mensaje

5.7 Módulo de IA / Asistente (`ai/`)

Objetivo: El cerebro conversacional de Capataz. Entiende lo que el usuario dice en lenguaje natural y lo traduce en acciones operativas.

Capacidades del asistente en Etapa 1:

Intención	Ejemplo del usuario	Acción del sistema
`task.create`	"Recuérdale a Juan que limpie la cocina a las 3"	Crea tarea asignada a Juan con due_date
`task.list`	"¿Qué pendientes hay hoy?"	Lista tareas pending del día
`task.complete`	"Ya terminé el inventario"	Marca tarea como completada
`checklist.start`	"Voy a abrir el local"	Inicia checklist de apertura
`checklist.complete`	"Listo" (durante checklist activo)	Marca item actual y pasa al siguiente
`reminder.create`	"Mañana a las 7 recuérdame pedir el pedido"	Crea recordatorio
`report.daily`	"¿Cómo vamos hoy?"	Genera resumen del día
`employee.status`	"¿Qué hizo María hoy?"	Muestra tareas completadas por María
`help`	"¿Qué puedes hacer?"	Muestra capacidades
`greeting`	"Buenos días"	Saludo + resumen rápido del día
`smalltalk`	"Gracias", "ok"	Respuesta empática contextual

Arquitectura del AI Engine:

```
Mensaje del usuario
  │
  ▼
┌──────────────────┐
│ Intent Classifier│  ← Clasificación rápida (reglas + LLM ligero)
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ Context Loader   │  ← Carga: negocio, empleados, tareas activas, hora
└──────────────────┘
         │
         ▼
┌──────────────────┐
│  Action Router   │  ← Decide si ejecuta acción o solo responde
└──────────────────┘
         │
    ┌────┴────┐
    ▼         ▼
┌─────────┐ ┌──────────┐
│Execute  │ │ Generate │
│ Action  │ │ Response │
│(DB ops)│ │ (vLLM)   │
└─────────┘ └──────────┘
     │          │
     └──────────┘
          ▼
   Respuesta al usuario
```

Prompt del sistema (concepto):

El asistente debe comportarse como un capataz amigable, directo y confiable. Habla en español mexicano coloquial pero profesional. No usa lenguaje técnico. Es breve — respuestas de 1-3 oraciones máximo. Siempre confirma las acciones que realizó. Si no entiende, pregunta una sola cosa para clarificar. Nunca juzga, nunca regaña, nunca es condescendiente. Conoce el contexto del negocio, los empleados y las tareas activas.

5.8 Módulo de Reportes (`reports/`)

Objetivo: Dar visibilidad al dueño sobre cómo va su negocio operativamente, sin necesidad de dashboards complejos.

Funcionalidades:

- Resumen diario automático (enviado por WhatsApp al cierre)
- Resumen semanal con tendencias básicas

- Consulta bajo demanda ("¿cómo vamos?")
- Métricas simples: tareas completadas vs pendientes, checklists hechos, actividad por empleado

Reportes disponibles en MVP:

Reporte	Frecuencia	Canal	Contenido
Resumen matutino	Diario	WhatsApp	Tareas del día, recordatorios, checklists pendientes
Cierre del día	Diario	WhatsApp	Tareas completadas, pendientes, vencidas
Resumen semanal	Lunes	WhatsApp	Tendencia de la semana, top empleados, áreas de mejora
Bajo demanda	Cuando lo pida	WhatsApp / App	Lo que el usuario pregunte

5.9 Módulo de Dashboard Web (`dashboard/`)

Objetivo: Vista web para el dueño del negocio donde puede ver todo de un vistazo sin depender de WhatsApp. Complementa al asistente, no lo reemplaza.

Stack del dashboard:

- **Server-side rendering:** FastAPI sirve HTML con Jinja2 templates.
- **Interactividad:** HTMX para actualizaciones parciales sin recargar página (ej: marcar tarea como completada, filtrar por fecha).
- **Gráficas:** Chart.js para visualizaciones (tareas completadas vs pendientes, actividad por día, rendimiento por empleado).
- **Estilos:** CSS vanilla con custom properties (variables CSS), sin frameworks de CSS. Diseño limpio y propio.
- **JS:** Vanilla JS para lógica del cliente. Cero dependencias de frameworks.

Vistas del dashboard:

Vista	Ruta	Contenido
Home	`/dashboard/`	Resumen del día: tareas pendientes, completadas, vencidas. Gráfica de actividad de la semana. Alertas y sugerencias de IA
Tareas	`/dashboard/tasks`	Lista de tareas con filtros (estado, asignado, categoría, fecha). Crear/editar tarea. Vista kanban opcional (pending → in_progress → completed)
Equipo	`/dashboard/team`	Lista de empleados con actividad reciente. Tareas asignadas y completadas por persona. Invitar nuevo miembro
Checklists	`/dashboard/checklists`	Checklists activos y completados. Historial de completaciones. Crear/editar checklist
Reportes	`/dashboard/reports`	Gráficas semanales/mensuales. Tendencias de tareas. Actividad por empleado. Exportar a PDF
Configuración	`/dashboard/settings`	Datos del negocio, horarios, categorías de tareas, preferencias de notificación
Login	`/dashboard/login`	Acceso con email + contraseña o con OTP por teléfono

Patrón de interactividad con HTMX:

```html
<!-- Ejemplo: marcar tarea como completada sin recargar -->
<button hx-post="/api/v1/businesses/{id}/tasks/{task_id}/complete"
        hx-target="#task-{task_id}"
        hx-swap="outerHTML"
        class="btn-complete">
  ✓ Completar
</button>

<!-- Ejemplo: filtrar tareas por estado -->
<select hx-get="/dashboard/tasks/partials/list"
        hx-target="#tasks-list"
        hx-trigger="change"
        name="status">
  <option value="all">Todas</option>
  <option value="pending">Pendientes</option>
  <option value="overdue">Vencidas</option>
  <option value="completed">Completadas</option>
</select>
```

Endpoints del dashboard (renderizan HTML, no JSON):

Método	Ruta	Respuesta
GET	`/dashboard/`	HTML — vista home
GET	`/dashboard/tasks`	HTML — vista de tareas
GET	`/dashboard/tasks/partials/list`	HTML parcial — solo la lista de tareas (para HTMX)
GET	`/dashboard/team`	HTML — vista de equipo
GET	`/dashboard/checklists`	HTML — vista de checklists
GET	`/dashboard/reports`	HTML — vista de reportes
GET	`/dashboard/settings`	HTML — configuración
GET	`/dashboard/login`	HTML — login
POST	`/dashboard/login`	Redirect — procesa login

Notas de implementación:

- El dashboard consume los mismos endpoints de API (`/api/v1/...`) internamente. No duplica lógica de negocio.

- Las vistas parciales (partials) permiten que HTMX actualice solo fragmentos de la página sin full reload.
- Chart.js se inicializa con datos inyectados en el template como JSON en un tag `<script>`.
- El diseño debe ser responsive (mobile-friendly) porque el dueño puede abrir el dashboard desde su celular.
- Autenticación por session cookie (HTTPOnly, Secure) para el dashboard web, separada del JWT de la API.

5.10 Módulo de Onboarding (`onboarding/`)

Objetivo: Que un negocio nuevo esté operando con Capataz en menos de 10 minutos, sin ayuda externa.

Flujo de onboarding (conversacional por WhatsApp):

```
Paso 1: "¡Hola! Soy Capataz, tu asistente de negocio. ¿Cómo se llama tu
negocio?"
  → Registra nombre del negocio

Paso 2: "¿A qué se dedica? (Restaurante / Tienda / Taller / Servicios / Otro)"
  → Registra giro y carga plantillas relevantes

Paso 3: "¿Cuántas personas trabajan contigo?"
  → Registra employee_count

Paso 4: "¿A qué hora abres y a qué hora cierras?"
  → Registra horarios operativos

Paso 5: "¡Listo! Ya estás configurado. Te sugiero estas tareas para hoy:
  [Lista de 3-5 tareas sugeridas según giro]
  ¿Quieres que las active?"
  → Crea tareas iniciales

Paso 6: "¿Quieres agregar a alguien de tu equipo? Mándame su número."
  → Invita empleados
```

Criterio de éxito: Onboarding completado en ≤ 10 minutos, al menos 1 tarea creada.

---

6. User Stories

6.1 Épica: Registro y Configuración

ID	Como...	Quiero...	Para...	Prioridad	Criterio de aceptación
US-001	Dueño de negocio	Registrarme con mi número de WhatsApp	Empezar a usar Capataz sin procesos complicados	P0	Recibo OTP por WhatsApp, lo verifico y quedo registrado en < 2 min
US-002	Dueño de negocio	Configurar mi negocio respondiendo preguntas simples	Que Capataz entienda mi contexto	P0	Onboarding completo en ≤ 10 min con nombre, giro, horarios y equipo
US-003	Dueño de negocio	Invitar a mis empleados mandando su número	Que ellos también reciban tareas	P1	Empleado recibe mensaje de bienvenida y queda vinculado al negocio
US-004	Empleado	Unirme al negocio de mi jefe	Recibir mis tareas por WhatsApp	P1	Confirmo con un "Sí" y quedo activo

6.2 Épica: Gestión de Tareas

ID	Como...	Quiero...	Para...	Prioridad	Criterio de aceptación
US-010	Dueño	Crear una tarea diciéndosela a Capataz por WhatsApp	No perder tiempo en interfaces	P0	Digo "Dile a Juan que haga inventario mañana a las 9" y Capataz crea la tarea correctamente
US-011	Dueño	Ver qué pendientes hay hoy	Saber qué falta por hacer	P0	Pregunto "¿Qué hay pendiente?" y recibo lista clara
US-012	Empleado	Ver mis tareas del día	Saber qué me toca hacer	P0	Escribo "¿Qué me toca hoy?" y recibo mi lista
US-013	Empleado	Marcar una tarea como terminada	Que mi jefe sepa que ya la hice	P0	Digo "Ya terminé [tarea]" y Capataz la marca completada con timestamp
US-014	Dueño	Recibir aviso cuando una tarea se vence	Actuar a tiempo	P0	Recibo notificación automática 2h antes y al vencer
US-015	Dueño	Crear tareas que se repitan	No tener que crear la misma tarea cada día	P1	Configuro "limpieza de baños todos los días a las 10" y se genera automáticamente
US-016	Dueño	Que Capataz me sugiera tareas según mi giro	No empezar de cero	P1	Al completar onboarding de restaurante, me sugiere: revisar inventario, checklist de apertura, revisar reseñas

6.3 Épica: Checklists

ID	Como...	Quiero...	Para...	Prioridad	Criterio de aceptación
US-020	Dueño	Crear un checklist de apertura	Que nadie olvide los pasos	P1	Creo checklist con items ordenados, se activa diario al abrir
US-021	Empleado	Completar el checklist paso a paso por WhatsApp	Hacerlo sin complicaciones	P1	Capataz me envía item por item, marco cada uno y al final confirma completado
US-022	Dueño	Ver si el checklist de hoy se completó	Saber que todo está en orden	P1	Pregunto "¿Se hizo la apertura?" y Capataz me dice quién lo hizo y a qué hora
US-023	Dueño	Usar plantillas de checklists para mi giro	No inventar todo desde cero	P2	Al seleccionar "restaurante", veo plantillas de apertura, cierre, limpieza

6.4 Épica: Recordatorios

ID	Como...	Quiero...	Para...	Prioridad	Criterio de aceptación
US-030	Dueño	Pedir un recordatorio en lenguaje natural	No olvidar cosas importantes	P0	"Recuérdame mañana a las 7 pedir el pedido de tortillas" → recordatorio creado
US-031	Dueño	Recibir un resumen al inicio del día	Arrancar organizado	P1	Recibo automáticamente: tareas del día, checklists, recordatorios
US-032	Dueño	Recibir un cierre del día	Saber cómo nos fue	P1	Recibo: tareas completadas, pendientes, vencidas, logros

6.5 Épica: Reportes y Visibilidad

ID	Como...	Quiero...	Para...	Prioridad	Criterio de aceptació
US-040	Dueño	Preguntar "¿cómo vamos?"	Tener un pulso rápido del negocio	P1	Capataz responde con resumen: tareas hoy (completadas/pendiente checklists, actividad
US-041	Dueño	Saber qué hizo cada empleado	Reconocer o corregir sin micromanagement	P2	"¿Qué hizo María hoy?" → lista de tareas completadas por María
US-042	Dueño	Recibir un resumen semanal	Ver tendencias sin pedirlo	P2	Cada lunes recibo: comparativa con seman anterior, top performers, áreas con más tareas vencidas

6.6 Épica: Asistente Conversacional

ID	Como...	Quiero...	Para...	Prioridad	Criterio de aceptación
US-050	Usuario	Hablar con Capataz en español coloquial	No tener que aprender comandos	P0	"Oye ponle a la Martha que cheque el refrigerador" → tarea creada y asignada correctamente
US-051	Usuario	Que Capataz entienda audio de WhatsApp	No tener que escribir	P2	Mando nota de voz, Capataz la transcribe y actúa según la intención
US-052	Usuario	Que Capataz me salude con contexto	Sentir que entiende mi negocio	P1	"Buenos días" → "Buenos días Luis. Hoy tienes 4 pendientes y el checklist de apertura no se ha hecho"
US-053	Dueño	Que Capataz me sugiera mejoras	Mejorar sin tener que pensarlo todo	P2	Después de 2 semanas de uso, Capataz sugiere: "He notado que los martes se vencen más tareas. ¿Quieres que mande los recordatorios más temprano ese día?"

---

7. Roadmap: Fases y Sprints

> Sprints de 2 semanas. Cada fase agrupa sprints por objetivo estratégico.

Fase 0: Fundaciones (Semanas 1–2)

Objetivo: Infraestructura base lista para desarrollar features.

Sprint 0 (Semanas 1–2)

Ticket	Entregable	Responsable
S0-01	Proyecto FastAPI configurado (estructura, config, CI/CD, scripts de deploy)	Dev Lead
S0-02	PostgreSQL + modelos base + migraciones Alembic (users, businesses, tasks)	Backend
S0-03	Auth: registro/login OTP por teléfono + JWT (access + refresh)	Backend
S0-04	Webhook de WhatsApp: recepción y envío de mensajes básicos (echo bot)	Backend
S0-05	Configurar Groq Cloud **pagado** como LLM temporal (OpenAI-compatible API, sin límites de contexto)	Dev Lead
S0-06	Testing framework (pytest + fixtures + test DB)	Backend

Criterio de salida: Un usuario puede registrarse por WhatsApp, enviar un mensaje y recibir un eco. LLM vía Groq responde a un prompt de prueba.

Fase 1: MVP Core (Semanas 3–8)

Objetivo: Un dueño puede registrar su negocio, crear tareas, asignarlas y dar seguimiento — todo por WhatsApp. Primeros pilotos.

Sprint 1 (Semanas 3–4): Onboarding + Tareas

Ticket	Entregable	User Stories	Responsable
S1-01	Onboarding conversacional completo por WhatsApp	US-002	Backend + AI
S1-02	CRUD de tareas vía API (endpoints completos con filtros)	US-010, US-011	Backend
S1-03	AI Engine v1: clasificador de intenciones + crear tarea por lenguaje natural	US-010, US-050	AI
S1-04	Modelo de negocio + vinculación owner	US-002	Backend
S1-05	Diseño de identidad visual para dashboard y WhatsApp	—	Diseñador
S1-06	Templates de WhatsApp enviados a aprobación de Meta	—	Dev Lead

Criterio de salida: Un dueño hace onboarding por WhatsApp y dice "ponle a Juan que haga inventario mañana" → Capataz crea la tarea.

Sprint 2 (Semanas 5–6): Equipo + Completar Tareas

Ticket	Entregable	User Stories	Responsable
S2-01	Invitar empleados por teléfono + aceptación por WhatsApp	US-003, US-004	Backend
S2-02	Empleado ve sus tareas ("¿Qué me toca hoy?")	US-012	Backend + AI
S2-03	Completar tareas por WhatsApp ("ya terminé")	US-013	Backend + AI
S2-04	Recordatorios manuales por lenguaje natural	US-030	Backend + AI
S2-05	Notificaciones automáticas de tareas por vencer / vencidas	US-014	Backend
S2-06	Diseño de templates HTML para dashboard (base layout, componentes)	—	Diseñador

Criterio de salida: Flujo completo dueño → crea tarea → empleado la ve → la completa → dueño recibe notificación. **Listo para pilotos.**

Sprint 3 (Semanas 7–8): Pilotos + Server Local

Ticket	Entregable	User Stories	Responsable
S3-01	Tareas recurrentes (diarias, semanales, auto-generación)	US-015	Backend
S3-02	Sugerencias de tareas por giro	US-016	AI
S3-03	**Servers operativos: Server 1 (Z6 G4 + 2x PRO 5000, texto) + Server 2 (Z6 G4 + 1x PRO 5000, OCR)**	—	Dev Lead
S3-04	**Migrar inferencia de Groq → vLLM local** (cambiar BASE_URL)	—	Dev Lead
S3-05	Onboarding de 5 negocios piloto en Atlixco	—	Dev Lead
S3-06	Recolección de feedback de pilotos	—	Todos

Criterio de salida: 5 negocios usando Capataz diariamente. 2 servers locales operativos (Server 1: texto, Server 2: OCR). Groq desconectado, inferencia 100% local.

Fase 2: Rutinas y Proactividad (Semanas 9–12)

Objetivo: Capataz se vuelve proactivo — no solo responde, inicia interacciones. Dashboard web operativo.

Sprint 4 (Semanas 9–10): Checklists + Dashboard v1

Ticket	Entregable	User Stories	Responsable
S4-01	CRUD de checklists + completación paso a paso por WhatsApp	US-020, US-021	Backend + AI
S4-02	Plantillas de checklists por giro (restaurante, tienda, taller)	US-023	Backend
S4-03	Resumen matutino automático ("Buenos días Luis, hoy tienes 4 pendientes...")	US-031, US-052	Backend + AI
S4-04	Dashboard web: auth (login con session cookie)	—	Backend
S4-05	Dashboard web: vista Home (resumen del día, gráfica de actividad)	—	Backend + Diseñador
S4-06	Dashboard web: vista Tareas (lista con filtros, CRUD vía HTMX)	—	Backend + Diseñador

Criterio de salida: Checklists funcionales por WhatsApp. Dashboard con login, home y tareas.

Sprint 5 (Semanas 11–12): Dashboard completo + Cierre del día

Ticket	Entregable	User Stories	Responsable
S5-01	Cierre del día automático (resumen de lo hecho / no hecho)	US-032	Backend + AI
S5-02	Verificar completación de checklist ("¿Se hizo la apertura?")	US-022	AI
S5-03	Dashboard web: vista Equipo (empleados, actividad, invitar)	—	Backend + Diseñador
S5-04	Dashboard web: vista Checklists (activos, historial)	—	Backend + Diseñador
S5-05	Dashboard web: vista Configuración (datos del negocio, horarios)	—	Backend
S5-06	**Procesamiento de imágenes vía WhatsApp → DeepSeek-OCR → texto estructurado** (fotos de inventarios, tickets, notas)	—	Backend + AI
S5-07	Escalar a 10+ negocios piloto	—	Dev Lead

Criterio de salida: Dashboard completo. 10+ negocios con interacción ≥ 5 días/semana.

Fase 3: Inteligencia y Reportes (Semanas 13–16)

Objetivo: Capataz muestra valor acumulado — el dueño ve que su negocio mejoró.

Sprint 6 (Semanas 13–14): Reportes

Ticket	Entregable	User Stories	Responsable
S6-01	Reporte bajo demanda por WhatsApp ("¿Cómo vamos?")	US-040	AI
S6-02	Actividad por empleado ("¿Qué hizo María hoy?")	US-041	AI
S6-03	Dashboard web: vista Reportes (gráficas semanales, tendencias)	—	Backend + Diseñador
S6-04	Exportar reportes a PDF desde dashboard	—	Backend
S6-05	Procesamiento de notas de voz (speech-to-text → acción)	US-051	Backend + AI
S6-06	Escalar a 20+ negocios	—	Dev Lead

Criterio de salida: Reportes funcionales. 20+ negocios activos.

Sprint 7 (Semanas 15–16): Proactividad + Refinamiento

Ticket	Entregable	User Stories	Responsable
S7-01	Reporte semanal automático (tendencias, comparativas)	US-042	Backend + AI
S7-02	Sugerencias proactivas de IA (detectar patrones, proponer mejoras)	US-053	AI
S7-03	Mejoras de NLP: español mexicano coloquial, corrección de errores	—	AI
S7-04	Optimización de performance (cache, queries, rate limiting)	—	Backend
S7-05	Bug fixes y mejoras de UX basadas en feedback de pilotos	—	Todos

Criterio de salida: Retención semanal > 60%. Al menos 3 testimonios de mejora operativa.

Fase 4: Escalamiento (Semanas 17–20)

Objetivo: Preparar para crecimiento más allá del piloto. App móvil.

Sprint 8 (Semanas 17–18): Multi-negocio + Integraciones

Ticket	Entregable	Responsable
S8-01	Multi-negocio (un owner gestiona más de un negocio)	Backend
S8-02	Categorías custom de tareas	Backend
S8-03	Integración Google Calendar (sync de tareas)	Backend
S8-04	Documentación técnica y de usuario	Dev Lead

Sprint 9 (Semanas 19–20): App Móvil v1

Ticket	Entregable	Responsable
S9-01	App Flutter: auth + vista principal (tareas del día)	Mobile Dev
S9-02	App Flutter: crear tarea, marcar completada	Mobile Dev
S9-03	App Flutter: notificaciones push	Mobile Dev + Backend
S9-04	App Flutter: resumen diario y semanal	Mobile Dev
S9-05	Testing y publicación en Play Store	Mobile Dev

Criterio de salida: Sistema estable para 50+ negocios. App Android publicada. Infraestructura lista para Etapa 2 (Data/Analytics).

---

8. Métricas de Validación y KPIs

8.1 Métricas de Adopción

Métrica	Definición	Meta MVP	Meta Fase 3
Negocios registrados	Total de negocios que completaron onboarding	10	30
Negocios activos (WAU)	Negocios con ≥ 1 interacción en la última semana	5	20
Tasa de completación de onboarding	% que completa onboarding una vez iniciado	> 70%	> 80%
Tiempo de onboarding	Minutos desde primer mensaje hasta negocio configurado	< 15 min	< 10 min
Empleados vinculados	Total de empleados activos en la plataforma	15	80

8.2 Métricas de Engagement

Métrica	Definición	Meta MVP	Meta Fase 3
DAU/WAU ratio	% de usuarios activos semanales que usan el sistema cada día	> 40%	> 60%
Mensajes por negocio/día	Promedio de mensajes enviados a Capataz por día	> 3	> 5
Tareas creadas/semana	Promedio por negocio	> 5	> 10
Tasa de completación de tareas	% de tareas creadas que llegan a completed	> 50%	> 65%
Checklists completados/semana	Promedio por negocio	N/A	> 3
Retención semana 1	% de negocios activos 7 días después de registro	> 60%	> 70%
Retención semana 4	% de negocios activos 28 días después de registro	> 30%	> 50%
Churn mensual	% de negocios que dejan de usar en un mes	< 30%	< 20%

8.3 Métricas de Calidad de IA

Métrica	Definición	Meta MVP	Meta Fase 3
Tasa de comprensión	% de mensajes donde la IA identifica correctamente la intención	> 75%	> 90%
Tasa de acción exitosa	% de intenciones que resultan en la acción correcta	> 70%	> 85%
Fallback rate	% de mensajes que caen en "no entendí"	< 25%	< 10%
Tiempo de respuesta	Latencia desde mensaje del usuario hasta respuesta	< 5 seg	< 3 seg

8.4 Métricas de Valor Percibido

Métrica	Definición	Meta Fase 3
NPS (Net Promoter Score)	Encuesta de recomendación	> 30
Tareas vencidas/semana (tendencia)	Reducción mes a mes	Tendencia descendente
Testimonios cualitativos	Dueños que reportan mejora en su operación	≥ 5
Referidos orgánicos	Negocios que llegan por recomendación	> 20% del total

8.5 Métricas Técnicas

Métrica	Definición	Meta
Uptime	Disponibilidad del servicio	> 99.5%
Error rate	% de requests que fallan (5xx)	< 1%
Webhook processing time	Tiempo de procesamiento de mensaje WhatsApp	p95 < 3 seg
DB query time	Tiempo promedio de queries principales	p95 < 100ms

---

9. Consideraciones Técnicas Críticas

9.1 WhatsApp Business Platform

- Se requiere una **cuenta de WhatsApp Business Platform** (antes WhatsApp Business API) con Meta.
- Los mensajes fuera de la ventana de 24 horas solo pueden enviarse con **message templates** pre-aprobados por Meta.
- Rate limits: respetar los límites de mensajería de Meta (varía según tier de la cuenta).
- **Templates necesarios en MVP:** saludo de bienvenida, recordatorio de tarea, resumen diario, invitación de empleado, OTP de verificación.
- Los templates deben ser aprobados por Meta antes del launch — planear con 1-2 semanas de anticipación.

9.2 Manejo de concurrencia

- Usar **async/await** nativo de FastAPI para I/O-bound operations.
- Cola de tareas (Celery/arq) para: envío de notificaciones, generación de reportes, procesamiento de audio, jobs de IA.
- Evitar bloquear el webhook de WhatsApp — procesar mensajes en background y responder rápido.

9.3 Seguridad y privacidad

- Datos en tránsito: HTTPS obligatorio (TLS 1.3).
- Datos en reposo: encripción a nivel de disco en el proveedor cloud.
- No almacenar datos sensibles innecesarios (sin datos financieros en Etapa 1).
- Cumplimiento con la Ley Federal de Protección de Datos Personales (LFPDPPP).
- Aviso de privacidad en el onboarding.
- Los mensajes de WhatsApp se almacenan para contexto del asistente, con política de retención.

9.4 Estrategia de IA / LLM (Self-Hosted con vLLM)

Decisión estratégica: Capataz AI corre su propio modelo de lenguaje en infraestructura propia usando vLLM como servidor de inferencia. Esto elimina dependencia de APIs comerciales, reduce costos a largo plazo y da control total sobre los datos.

Modelo mínimo objetivo: 16B de parámetros. Un modelo de 16B ofrece capacidad de razonamiento significativamente superior a 8B para tareas de NLP en español: mejor comprensión de contexto, menos alucinaciones y mayor precisión en clasificación de intenciones complejas.

Stack de IA:

Componente	Tecnología	Detalle
Servidor de inferencia	vLLM	API compatible con OpenAI, paged attention, batching continuo
Modelo de texto (GPU 0)	Qwen3-16B FP16 o Mistral-Nemo-16B	16B+ parámetros en precisión completa. ~32GB en VRAM. Español nativo
Modelo de visión/OCR (GPU 1)	DeepSeek-OCR (~3B params, ~7GB BF16)	OCR multimodal para procesar imágenes enviadas por WhatsApp: fotos de inventarios, notas, tickets, menús, documentos. Soportado nativamente en vLLM
Modelo futuro	Qwen2.5-32B FP16 o Llama 3.1-70B AWQ	96GB de VRAM permiten 70B cuantizado (~~38GB) o 32B en FP16 (~~64GB)
GPUs del proyecto	3x NVIDIA RTX PRO 5000 Blackwell (48GB GDDR7 ECC c/u) = 144GB total	$6,000 USD c/u (B&H). 2 en Server 1 (texto), 1 en Server 2 (OCR). Blower 300W, dual-slot, 24/7. Slot libre en Server 2 para 4ta GPU
Cuantización	No necesaria con PRO 5000 para modelos ≤24B	Con 48GB por GPU, modelos de 16B corren en FP16 completo sin pérdida
Embedding model	bge-m3 o all-MiniLM-L6-v2	Clasificación de intenciones y búsqueda semántica (corre en CPU o en GPU 1 junto con OCR)

Distribución de GPUs (2 servers):

```
SERVER 1 — HP Z6 G4 (dual Xeon, 192GB RAM, PSU 1000W)
├── GPU 0: RTX PRO 5000 (48GB) → Modelo de texto principal
│   └─ Qwen3-16B FP16 (~32GB VRAM)
│   └─ ~16GB libres para KV cache / batching
│
└── GPU 1: RTX PRO 5000 (48GB) → Segundo modelo de texto / escala
    └─ Réplica de Qwen3-16B (duplica throughput)
    └─ O tensor parallel GPU 0+1 para modelos 32B+ FP16
    └─ + Embeddings model bge-m3 (~1GB)

SERVER 2 — HP Z6 G4 (dual Xeon, 192GB RAM, PSU 1000W)
├── GPU 2: RTX PRO 5000 (48GB) → DeepSeek-OCR dedicado
│   └─ DeepSeek-OCR BF16 (~7GB) — procesamiento de imágenes
│   └─ ~41GB libres para batching masivo de imágenes
│   └─ Puerto 8001 (vLLM independiente)
│
└── [SLOT LIBRE] → 4ta RTX PRO 5000 cuando se necesite
    └─ Réplica de OCR para alto volumen de imágenes
    └─ O segundo modelo de texto para más throughput
    └─ O modelo de visión más grande (DeepSeek-VL2, etc.)
```

MVP (Sprint 3): 3x RTX PRO 5000 instaladas (2 en Server 1, 1 en Server 2).
Escala: Agregar 4ta GPU en Server 2 cuando el volumen lo requiera.

¿Por qué 2 servers separados?

- Aislamiento: si un server cae, el otro sigue operando (texto o OCR siguen vivos)
- Poder: cada Z6 G4 con PSU 1000W maneja cómodamente 2x 300W GPUs (~850W total)
- Escalabilidad: cada server tiene un slot PCIe x16 libre para una 4ta y 5ta GPU
- Mantenimiento: se puede actualizar un server sin tumbar el otro

¿Por qué DeepSeek-OCR en Capataz?

Los pequeños negocios mandan fotos constantemente por WhatsApp: inventarios escritos a mano, tickets de compra, notas en papel, menús, listas de precios, facturas. DeepSeek-OCR convierte todo eso en texto estructurado (Markdown) que Capataz puede procesar, almacenar y usar para inteligencia operativa. Casos de uso:

- Foto de inventario escrito a mano → tabla digitalizada
- Foto de ticket/factura → monto y proveedor extraídos
- Foto de menú → checklists de preparación auto-generados
- Foto de documento → texto buscable en el historial del negocio

Ruta de migración:

```
FASE 0-1 (Sprints 0-3):  Groq Cloud API PAGADO (sin límites de contexto)
                          └─ Modelo texto: llama-3.3-70b-versatile
                          └─ OCR: no disponible en Groq → Tesseract como puente
temporal
                          └─ Costo: ~$5,000 MXN/mes (plan pagado)

FASE 1+ (Sprint 3+):     Server 1: Z6 G4 + 2x RTX PRO 5000 (96GB)
                          └─ GPU 0: Qwen3-16B FP16 (texto)
                          └─ GPU 1: réplica o TP para modelos más grandes

                         Server 2: Z6 G4 + 1x RTX PRO 5000 (48GB)
                          └─ GPU 2: DeepSeek-OCR (visión) dedicado
                          └─ Slot libre para 4ta GPU

                          └─ Costo electricidad: ~$4,000 MXN/mes (2 servers)
                          └─ Groq queda como fallback de emergencia
```

¿Caben 2x RTX PRO 5000 Blackwell en cada HP Z6 G4?

Sí. Cada Z6 G4 maneja 2 GPUs cómodamente:

Aspecto	RTX PRO 5000	HP Z6 G4	¿Compatible?
Interfaz	PCIe 5.0 x16	PCIe 3.0 x16 (2 slots)	**Sí** — backward compatible. <3% diferencia en inferencia
Form factor	Dual-slot, blower	2 slots PCIe x16	**Sí** — caben las 2
TDP	300W × 2 = 600W	PSU 1000W	**Sí** — ~850W total con CPUs y storage
Conector	16-pin (12VHPWR)	Cables 6/8-pin	**Adaptador requerido** — 2 por server, 4 total
Largo	~267mm	Hasta 317mm	**Sí**

Para MVP: Server 1 con 2 GPUs (texto), Server 2 con 1 GPU (OCR). Slot libre en Server 2.

PCIe 3.0 vs 5.0 — impacto real para inferencia:

PCIe solo mueve datos CPU↔GPU: cargar el modelo al inicio (~32GB, una vez) y tokens durante inferencia (mínimo). El cuello de botella real es el bandwidth de VRAM (1,344 GB/s en la PRO 5000), no el bus PCIe. Benchmarks: **<3% diferencia** entre Gen 3 y Gen 5 para inferencia. No justifica cambiar de chasis.

Donde SÍ limita: tensor parallelism entre GPUs del mismo server (~15 GB/s PCIe vs ~300 GB/s NVLink). Mitigación: correr modelos que quepan en 1 GPU (hasta 48GB = ~24B FP16).

Arquitectura de inferencia (2 servers):

```
Mensaje del usuario (texto o imagen)
  │
  ▼
┌──────────────────────────┐
│  FastAPI (app en VPS)    │
│  ¿Es imagen? ──────────────────────────────────┐
│  ¿Es texto?  ──────────┐                       │
└──────────────────────────┘                       │
             │                                     │
             │ HTTP (LAN)                          │ HTTP (LAN)
             ▼                                     ▼
┌──────────── SERVER 1 (Z6 G4) ────────┐  ┌──── SERVER 2 (Z6 G4) ────┐
│                                      │  │                          │
│  vLLM :8000 (GPU 0)                  │  │  vLLM :8001 (GPU 2)      │
│  Qwen3-16B FP16                      │  │  DeepSeek-OCR BF16       │
│  RTX PRO 5000 (48GB)                 │  │  RTX PRO 5000 (48GB)     │
│  → Texto / NLP                       │  │  → Imágenes → Markdown   │
│                                      │  │                          │
│  vLLM :8002 (GPU 1) [réplica/TP]     │  │  [SLOT LIBRE → 4ta GPU]  │
│  RTX PRO 5000 (48GB)                 │  │                          │
└──────────────────────────────────────┘  └──────────────────────────┘
```

Configuración de vLLM (systemd services en cada server):

```
# === SERVER 1 (Z6 G4 — texto) ===

# /etc/systemd/system/vllm-text.service
[Unit]
Description=vLLM Text Model - Capataz AI (GPU 0)
After=network.target

[Service]
Type=simple
User=capataz
Environment=CUDA_VISIBLE_DEVICES=0
ExecStart=/home/capataz/vllm/venv/bin/python -m vllm.entrypoints.openai.api_server \
    --model /home/capataz/models/Qwen3-16B \
    --dtype float16 \
    --max-model-len 8192 \
    --gpu-memory-utilization 0.85 \
    --port 8000 \
    --api-key ${VLLM_API_KEY}
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

```
# === SERVER 2 (Z6 G4 — OCR) ===

# /etc/systemd/system/vllm-ocr.service
[Unit]
Description=vLLM DeepSeek-OCR - Capataz AI
After=network.target

[Service]
Type=simple
User=capataz
Environment=CUDA_VISIBLE_DEVICES=0
ExecStart=/home/capataz/vllm/venv/bin/python -m vllm.entrypoints.openai.api_server \
    --model deepseek-ai/DeepSeek-OCR \
    --dtype bfloat16 \
    --max-model-len 8192 \
    --gpu-memory-utilization 0.50 \
    --port 8001 \
    --api-key ${VLLM_API_KEY}
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

```
# Comandos de gestión
sudo systemctl start vllm
sudo systemctl enable vllm
sudo systemctl status vllm
journalctl -u vllm -f
```

Llamada desde FastAPI (clients apuntan a cada server por LAN):

```python
from openai import AsyncOpenAI

# Cliente para modelo de texto (Server 1, GPU 0)
text_client = AsyncOpenAI(
    base_url=settings.LLM_TEXT_URL,  # Groq: "https://api.groq.com/openai/v1"
                                     # Local: "http://server1:8000/v1"
    api_key=settings.LLM_API_KEY,
)

# Cliente para OCR (Server 2, GPU 2)
ocr_client = AsyncOpenAI(
    base_url=settings.LLM_OCR_URL,  # "http://server2:8001/v1"
    api_key=settings.LLM_API_KEY,
)
```

Estrategia de clasificación de intenciones:

- Implementar un **clasificador ligero** como primera capa (embeddings + cosine similarity o un modelo fine-tuned pequeño) para evitar enviar todos los mensajes al LLM.
- Solo escalar al LLM completo cuando la intención es ambigua o requiere generación de texto.
- Esto reduce la carga en GPU y mejora la latencia para intenciones simples.

Ventajas del enfoque self-hosted:

- Sin costo por token — costo fijo de electricidad.
- Control total sobre datos (ningún mensaje sale a terceros).
- Posibilidad de fine-tuning del modelo con datos propios.
- Sin rate limits externos.
- Latencia predecible y controlable.
- Con 96GB de VRAM (2x PRO 5000), capacidad para modelos de hasta 70B cuantizados.

Riesgos del enfoque self-hosted:

- Requiere conocimiento de DevOps/MLOps (mitigar: systemd + scripts simples).
- Si el server local se cae, no hay inferencia hasta que se repare (mitigar: Groq como fallback de emergencia, mantener API key activa).
- PCIe 3.0 limita tensor parallelism eficiente entre GPUs (mitigar: correr modelos que quepan en 1 GPU, usar segunda GPU como réplica).

- Adaptadores de poder 8-pin a 16-pin deben ser de calidad (mitigar: usar adaptadores oficiales o del fabricante del PSU).
- Mantener **prompts versionados** en código para iteración rápida.
- Log de todas las interacciones de IA para revisión, mejora y eventual fine-tuning.

9.5 Escalabilidad

- Diseñar para **multi-tenant** desde el inicio (business_id en todas las tablas).
- Paginación basada en cursor (no offset) para queries de listado.
- Connection pooling con asyncpg.
- Rate limiting por negocio para prevenir abuso.

---

10. Riesgos y Mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Baja adopción diaria por parte de los dueños	Alta	Crítico	Onboarding ultra-simple, resumen matutino proactivo, demostrar valor en la primera semana
La IA no entiende el español coloquial de los usuarios	Media	Alto	Dataset de entrenamiento con variantes mexicanas, fallback amigable "no entendí, ¿puedes decirlo de otra forma?", mejora continua con logs
Meta cambia políticas/precios de WhatsApp API	Media	Alto	Arquitectura modular de canales, tener SMS y app como fallback
El usuario siente que Capataz lo vigila	Media	Alto	Tono del asistente de ayuda (no de control), nunca reportar a terceros, transparencia en qué datos se guardan
Costo de GPU para LLM self-hosted	Media	Medio	3x RTX PRO 5000 a $6,000 USD c/u (B&H). Equipo propio del proyecto. Groq pagado como fallback de emergencia
Server local falla o se daña	Media	Medio	Con 2 servers: si uno cae, el otro sigue (texto O OCR siguen vivos). Groq como fallback para texto. UPS por server. Backups automáticos de DB
RTX PRO 5000 requiere adaptador 16-pin en Z6 G4	Baja	Medio	Usar adaptadores oficiales. Verificar conectores 8-pin del PSU antes de comprar
Empleados se resisten a participar	Media	Medio	Interacción mínima requerida, tono no invasivo, reconocimiento por tareas hechas
Problemas de conectividad en zonas de Atlixco	Baja	Medio	WhatsApp funciona con poco ancho de banda, mensajes cortos, procesamiento server-side

11. Definición de "Hecho" (Definition of Done)

Para considerar un feature como terminado:

1. **Código:** Escrito, revisado (code review) y mergeado a main.
2. **Tests:** Tests unitarios y de integración con cobertura > 80% para el feature.
3. **API docs:** Endpoint documentado automáticamente en OpenAPI (FastAPI lo hace por defecto).
4. **WhatsApp:** Si el feature tiene interacción por WhatsApp, probado end-to-end con mensaje real.
5. **Error handling:** Errores manejados con mensajes claros al usuario (no errores técnicos).
6. **Logging:** Eventos relevantes logueados para debugging y métricas.
7. **Performance:** Query time < 100ms (p95), response time < 3 seg para WhatsApp.
8. **Seguridad:** Sin exposición de datos entre negocios (multi-tenant validado).

12. Convenciones del Proyecto

12.1 Código

- Python: seguir PEP 8, usar type hints en todo, docstrings en funciones públicas.
- FastAPI: un router por módulo, schemas Pydantic separados (request/response).
- Naming: snake_case para todo en Python, kebab-case para URLs.
- Commits: conventional commits (`feat:`, `fix:`, `docs:`, `refactor:`).
- Branch strategy: `main` → `develop` → `feature/xxx`.

12.2 API

- Versionamiento: `/api/v1/...`
- Respuestas estándar: `{ "data": ..., "meta": { "page": ..., "total": ... } }`
- Errores estándar: `{ "error": { "code": "...", "message": "...", "details": [...] } }`
- HTTP status codes: 200 (ok), 201 (created), 400 (bad request), 401 (unauthorized), 403 (forbidden), 404 (not found), 422 (validation error), 500 (internal error).
- Paginación: cursor-based con `?cursor=xxx&limit=20`.

12.3 Base de datos

- UUIDs como primary keys (no auto-increment).
- Timestamps siempre en UTC con timezone (`TIMESTAMPTZ`).
- Soft deletes solo cuando sea necesario (usando `is_active` o `deleted_at`).
- Migraciones con Alembic — nunca modificar la DB manualmente.

---

13. Glosario

Término	Definición
Capataz	El asistente de IA conversacional que interactúa con el usuario
Negocio	La unidad organizacional principal — un restaurante, tienda, taller, etc.
Owner	Dueño del negocio, usuario principal con permisos completos
Member	Cualquier persona vinculada al negocio (owner, manager, employee)
Tarea	Unidad de trabajo asignable con estado, prioridad y fecha límite
Checklist	Lista de verificación con items ordenados para rutinas operativas
Recurrencia	Regla de repetición temporal (diario, semanal, mensual, custom)
Resumen	Reporte automático generado por IA sobre el estado del negocio
Hormiguero	Punto de activación territorial para onboarding presencial (futuro)
Canal	Medio de comunicación: WhatsApp, SMS, app, web
Intención	Lo que el usuario quiere lograr con su mensaje (crear tarea, consultar, etc.)
OTP	One-Time Password — código de verificación enviado por WhatsApp/SMS
Template (WhatsApp)	Mensaje pre-aprobado por Meta para enviar fuera de ventana de 24h

---

14. Próximos Pasos Inmediatos

1. **Configurar cuenta de WhatsApp Business Platform** con Meta (requiere verificación de negocio — puede tomar 1-2 semanas).
2. **Inicializar repositorio** con estructura FastAPI, scripts de deploy, CI/CD y test framework.
3. **Diseñar y ejecutar migraciones** del modelo de datos inicial.
4. **Configurar Groq Cloud** como LLM temporal y validar con prompts en español. Definir `LLM_BASE_URL` como variable de entorno para el switch a vLLM local.
5. **Implementar auth + webhook de WhatsApp** como primer vertical funcional (Sprint 0).
6. **Definir y documentar los templates de WhatsApp** que se necesitan para MVP y enviarlos a aprobación de Meta.
7. **Preparar 2x HP Z6 G4:** instalar Ubuntu 24.04 LTS, drivers NVIDIA, CUDA toolkit, vLLM en ambos. Server 1: instalar 2x RTX PRO 5000. Server 2: instalar 1x RTX PRO 5000 + dejar slot libre.
8. **Reclutar 5 negocios piloto** en Atlixco para testing de Fase 1.
9. **Validar DeepSeek-OCR** con fotos reales de inventarios, tickets y notas de negocios piloto.

Este documento es un artefacto vivo. Debe actualizarse conforme el equipo de desarrollo avance y se obtenga feedback de los pilotos.